EcoSync – Ciberseguridad (3ra entrega)

Nov 6, 2026 · @joaquin

1. Introducción

En esta tercera entrega la ciberseguridad de SiGeRU pasa de estar planificada a estar implementada y probada en el sistema. La letra pide tener en cuenta la seguridad "tanto a nivel de software como a nivel de telecomunicaciones" y "elaborar estrategias de seguridad en cada capa, complementaria una de otra". Este documento muestra qué se hizo en cada capa, qué riesgo cubre y cómo se comprobó que funciona.

Equipo EcoSync: Valentín Ramírez (Coordinador), Joaquín Ximénez (Subcoordinador) y Juan Cuadro.

Qué cambió desde la 2da entrega

En la 2da entrega	En la 3ra entrega
Amenazas, vulnerabilidades y controles descriptos	Controles implementados en el código y en la base, con tests automáticos
"Falta de trazabilidad" como amenaza	Tabla `auditoria` + página **Seguridad** para el administrador
Fail2ban solo para SSH	También bloqueo por intentos fallidos en el login de SiGeRU
Política de contraseñas general	Política aplicada en el sistema: 8 caracteres mínimo, letras y números
Sesión sin vencimiento	La sesión se cierra sola a los 30 minutos sin uso
La base se usaba con root	Usuario `sigeru_app` con permisos mínimos para las APIs
Archivos internos accesibles desde el navegador	Solo `api.php` se puede abrir; tests y configuración bloqueados (403)
Solo las contraseñas protegidas (bcrypt)	Teléfono cifrado con AES-256-GCM y código de verificación guardado como hash SHA-256
HTTPS planificado	Certificado SSL de la CA EcoSync funcionando, solo TLS 1.2 y 1.3

En la 2da entrega	En la 3ra entrega
Backups sin cifrar	Backups con hash SHA-256, cifrado AES-256 y firma digital RSA
Sin revisión OWASP	Informe OWASP Top 10 con 28 pruebas automáticas que pasan

Nota: en la documentación de Sistemas Operativos aparecen Nginx y PostgreSQL, pero el sistema funciona con **Apache y MySQL/MariaDB** (lo que pide la letra). Este documento usa lo que realmente está implementado.

Requisitos de la tercera entrega (Requisitos de Ciberseguridad 2026, Prof. Gabriela Borba) y dónde se cumple cada uno:

Requisito	Sección	Implementación en el repositorio
Hashing, cifrado y firma digital implementados + documento de especificación	2	`sigeru-api-usuarios/cifrado.php`, `modelos/Usuario.php`, `seguridad/respaldos/`
Certificado SSL dentro del servidor	3	`seguridad/ssl/` (CA propia + certificado del servidor, Apache con TLS 1.2 y 1.3)
Políticas de seguridad de usuario	4	Política de contraseñas, bloqueo, sesiones, roles y auditoría aplicados en las APIs
Informe OWASP de aspectos de seguridad del sitio	5	`seguridad/owasp/revision_owasp.sh` (28 pruebas, todas pasan)

Las secciones 6 a 13 son la base de seguridad del sistema (capas, riesgos, red, base de datos, respuesta a incidentes y pruebas) y retoman lo pedido en la 1ra y 2da entrega.

2. Especificación de hashing, cifrado y firma digital

SiGeRU usa siete mecanismos criptográficos, todos implementados y probados: dos de hashing, tres de cifrado y dos de firma digital. Se usaron las funciones de PHP (`password_hash`, `hash`, `openssl_*`) y la herramienta `openssl` del servidor, sin inventar algoritmos propios.

#	Tipo	Algoritmo	Qué protege	Dónde está	Prueba
H1	Hashing	bcrypt (costo 10, sal al azar)	Contraseñas de los usuarios	`password_hash()` en `modelos/Usuario.php`	Tests de modelos: no se guarda el texto, `password_verify` da bien
H2	Hashing	SHA-256	Código del enlace de verificación del correo; integridad de los respaldos	`resumen()` en `cifrado.php`; `sha256sum` en `respaldo_firmado.sh`	En la base hay 64 caracteres hex, no el código
C1	Cifrado simétrico	AES-256-GCM (IV de 96 bits al azar, etiqueta de 128 bits)	Teléfono de los usuarios en la base	`cifrar()` / `descifrar()` en `cifrado.php`	5 tests: no se ve el número, se descifra, IV distinto, detecta cambios
C2	Cifrado simétrico	AES-256-CBC con clave derivada por PBKDF2 (200.000 vueltas, sal)	Respaldos de la base	`respaldo_firmado.sh`	El respaldo se abre solo con `clave_respaldo`
C3	Cifrado en tránsito	TLS 1.2 / 1.3 (ECDHE + AES-256-GCM)	Todo lo que viaja entre el navegador y el servidor; correos (STARTTLS)	Apache `mod_ssl` (sección 3); PHPMailer	`openssl s_client`
F1	Firma digital	RSA 3072 + SHA-256	Respaldos: que los hizo nuestro servidor y nadie los cambió	`respaldo_firmado.sh` firma, `verificar_respaldo.sh` comprueba	Un byte cambiado: "Verification failure"
F2	Firma digital	RSA 4096 (CA) firma con SHA-256 un certificado RSA 2048	Identidad del servidor en HTTPS	`crear_certificados.sh`	`openssl verify`: OK

2.1 Hashing

Un hash es de un solo sentido: no se puede volver al dato original, solo comparar. Por eso se usa para cosas que hay que **comprobar** pero no **leer**.

- **Contraseñas (bcrypt)**: `password_hash($contrasena, PASSWORD_DEFAULT)` genera una sal distinta para cada usuario y repite el cálculo 2^10 veces, así probar millones de contraseñas es lento. En el login se usa `password_verify()`. Se eligió bcrypt y no SHA-256 porque SHA-256 es muy rápido y eso ayuda a quien intenta adivinar.
- **Código de verificación (SHA-256)**: el enlace del correo lleva un código de 32 caracteres hecho con `random_bytes(16)`. En la base se guarda `hash("sha256", codigo)`. Cuando el vecino abre el enlace se calcula el hash de lo que trae y se busca ese hash. Si alguien ve la base, no puede armar el enlace y verificar una cuenta ajena.
- **Respaldos (SHA-256)**: al hacer el respaldo se guarda su `sha256sum`; al restaurarlo se vuelve a calcular y tiene que dar igual.

2.2 Cifrado

El cifrado sí es de ida y vuelta: con la clave se recupera el dato. Se usa para lo que hay que **leer** después.

Teléfono (AES-256-GCM). Es un dato personal (Ley 18.331 de Protección de Datos Personales). Cada vez que se guarda un teléfono:

1. Se genera un IV de 12 bytes al azar (`random_bytes(12)`), así dos teléfonos iguales quedan distintos en la base.
2. `openssl_encrypt($telefono, "aes-256-gcm", $clave, OPENSSL_RAW_DATA, $iv, $etiqueta)` devuelve el texto cifrado y una etiqueta de 16 bytes.
3. En la columna `telefono` se guarda `enc:` + base64(IV + etiqueta + cifrado), por ejemplo `enc:0LuUXmtS2bVKMGcaznoSWyS...`

Al leer se hace al revés con `openssl_decrypt`. Si alguien cambia un solo carácter en la base, la etiqueta no coincide y el sistema muestra "(no se pudo descifrar)" en vez de un dato falso. Los teléfonos que estaban sin cifrar se cifran solos la primera vez que el administrador abre Usuarios.

Respaldos (AES-256-CBC + PBKDF2): `openssl enc -aes-256-cbc -pbkdf2 -iter 200000 -salt`. El archivo sin cifrar se borra después de cifrarlo, así un respaldo robado no sirve sin la clave.

En tránsito (TLS): ver sección 3.

2.3 Firma digital

La firma digital usa un par de claves: con la **privada** se firma y con la **pública** cualquiera comprueba. Prueba dos cosas: quién lo hizo (autenticidad) y que nadie lo cambió después (integridad).

- **Respaldos**: MV2 firma cada respaldo cifrado con `openssl dgst -sha256 -sign firma_privada.pem` (RSA 3072). MV3 y quien restaure solo tienen `firma_publica.pem` y comprueban con `openssl dgst -sha256 -verify`. Si la firma no da, `verificar_respaldo.sh` no restaura nada. Se probó cambiando 1 byte del archivo: la verificación falló.
- **Certificado SSL**: la CA "EcoSync CA" firma el certificado del servidor. El navegador comprueba esa firma para saber que está hablando con el servidor de verdad (sección 3).

2.4 Gestión de claves

Clave	Tamaño	Dónde se guarda	Quién la puede leer
Clave de cifrado de datos (`sigeru_clave.php`)	256 bits	Fuera de la carpeta web y del repositorio; en Docker un volumen aparte; o variable `SIGERU_CLAVE`	El usuario de Apache
`firma_privada.pem`	RSA 3072	`/etc/sigeru` en MV2	Solo root (600)
`clave_respaldo`	48 bytes al azar	`/etc/sigeru` en MV2, copia guardada aparte	Solo root (600)
`ca.key` y `sigeru.key`	RSA 4096 y 2048	`/etc/ssl/sigeru` en MV1	Solo root (600)

Ninguna clave está en GitHub. Si la clave de datos se pierde, los teléfonos no se pueden recuperar: por eso se respalda junto con `clave_respaldo`, en un lugar distinto de los respaldos.

3. Certificado SSL en el servidor

El servidor Apache de SiGeRU atiende por HTTPS con un certificado firmado por una autoridad certificadora propia, **EcoSync CA**, y solo acepta TLS 1.2 y 1.3. Se eligió una CA propia porque el sistema funciona en la red interna de la Intendencia (192.168.1.0/24), donde Let's Encrypt no puede validar el dominio. Si se publica con un dominio, se usa `certbot --apache` y el resto de la configuración queda igual.

3.1 Cómo se generó

`seguridad/ssl/crear_certificados.sh sigeru.local 192.168.1.200`:

1. Crea la clave de la CA (RSA 4096) y su certificado autofirmado, válido 10 años.
2. Crea la clave del servidor (RSA 2048) y un pedido de firma (CSR) con `CN=sigeru.local`.
3. La CA firma el certificado con SHA-256, válido 825 días, con los nombres `sigeru.local`, `localhost`, `192.168.1.200` y `127.0.0.1` (subjectAltName) y uso `serverAuth`.
4. Deja las claves privadas con permisos 600 y comprueba la firma con `openssl verify`.

3.2 Configuración de Apache

`seguridad/ssl/sigeru-ssl.conf`: el puerto 80 redirige a HTTPS con un 301; en el 443 `SSLProtocol -all +TLSv1.2 +TLSv1.3`, solo cifrados ECDHE con AES-GCM o ChaCha20 (secreto perfecto hacia adelante) y `Strict-Transport-Security: max-age=31536000`. Cuando la conexión es HTTPS, la API marca sola la cookie de sesión como `Secure`.

En XAMPP se usa el mismo script desde Git Bash y se cambian las dos líneas del certificado en `httpd-ssl.conf` (pasos en `seguridad/LEEME.md`).

3.3 Evidencia (servidor de pruebas, 06/11/2026)

Comprobación	Resultado
`openssl verify -CAfile ca.crt sigeru.crt`	`sigeru.crt: OK`
Emisor y sujeto	`issuer: O=EcoSync, CN=EcoSync CA` · `subject: O=EcoSync, CN=sigeru.local`
Algoritmo de firma y clave	`sha256WithRSAEncryption`, clave pública de 2048 bits
Vigencia	06/11/2026 al 08/02/2029
Conexión TLS 1.3	`TLSv1.3`, cifrado `TLS_AES_256_GCM_SHA384`, verificación del certificado: `0 (ok)`
Conexión TLS 1.2	`TLSv1.2`, cifrado `ECDHE-RSA-AES256-GCM-SHA384`, `0 (ok)`
Cabecera HSTS	`Strict-Transport-Security: max-age=31536000`
Cookie por HTTPS	`SIGERU=...; path=/; secure; HttpOnly; SameSite=Strict`

Comprobación	Resultado
API por HTTPS	`https://localhost/sigeru-api-usuarios/api.php/sesion` responde 401 en JSON (anda, sin sesión)

TLS 1.0 y 1.1 quedan desactivados por `SSLProtocol`. Para la defensa conviene agregar una captura del candado del navegador con el certificado abierto, con `ca.crt` instalado en la PC.

4. Políticas de seguridad de usuario

Estas son las reglas que todo usuario de SiGeRU tiene que cumplir y que el sistema hace cumplir solo. Cada política dice qué se pide y cómo lo controla el sistema.

4.1 Alta de cuentas

- Los vecinos se registran solos desde la landing: tienen que verificar el correo con un enlace de un solo uso y después un administrador aprueba la cuenta. Hasta eso no pueden entrar.
- Las cuentas de Operario, Recolector y Administrador las crea solo un administrador. Nadie puede elegir su propio rol al registrarse (la API lo fuerza a Vecino).
- Una cuenta por persona y por correo: el correo no se puede repetir.

4.2 Contraseñas

Regla	Cómo lo controla el sistema
Al menos 8 caracteres, con letras y números (máximo 72)	Formulario (`pattern`) y API (`revisarContrasena()`), responde 400 si no cumple
No se guarda en texto, solo el hash bcrypt	`password_hash()`
No se comparte ni se reutiliza la de otros servicios	Política escrita (se informa al dar el alta)
Cambiarla si hay sospecha de que otro la sabe	El administrador la cambia desde Usuarios y queda en la auditoría
Nadie del equipo pide la contraseña por correo ni por teléfono	Política escrita

Los usuarios de prueba (`123456` , `admin123`) son solo para la defensa: se borran antes de usar el sistema con datos reales.

4.3 Intentos fallidos y bloqueo

Con 5 contraseñas equivocadas en 15 minutos el correo se bloquea 15 minutos (respuesta 429). Los dos números se cambian en Parámetros. Los intentos y los bloqueos quedan en la auditoría con la IP.

4.4 Sesiones

- La sesión se cierra sola después de 30 minutos sin usar el sistema.
- Al terminar hay que usar el botón de cerrar sesión, sobre todo en computadoras compartidas.
- La cookie de sesión no se puede leer desde JavaScript, no viaja desde otras páginas y con HTTPS solo viaja cifrada.

4.5 Permisos por rol (mínimo privilegio)

Cada rol ve y hace solo lo que necesita para su trabajo:

Rol	Puede	No puede
Vecino	Reportar incidencias y ver las suyas, ver el mapa	Ver incidencias de otros, datos de otros usuarios, gestión
Recolector	Ver su recorrido y atender las incidencias de su cuadrilla	Crear o borrar datos de gestión, usuarios
Operario	Gestionar contenedores, camiones, recorridos, centros, repuestos, incidencias e informes	Usuarios, roles, parámetros, auditoría
Administrador	Todo, incluido usuarios, roles, parámetros y auditoría	Cambiarse su propio rol o borrarse a sí mismo

Los cambios de rol quedan en la auditoría (quién, a quién, de qué rol a qué rol).

4.6 Datos personales (Ley 18.331)

- Se piden solo los datos necesarios: nombre, correo, teléfono (opcional) y dirección del vecino.
- El teléfono se guarda cifrado y las contraseñas como hash.

- Solo el administrador ve la lista de usuarios. El personal ve el nombre de quien reportó una incidencia, no sus datos de contacto.
- Si un vecino pide que se borren sus datos, el administrador borra la cuenta; sus incidencias quedan sin nombre.

4.7 Baja y revisión de cuentas

- Cuando alguien deja de trabajar en la Intendencia, el administrador le cambia el rol o borra la cuenta ese mismo día.
- Una vez por mes el administrador revisa en la página Seguridad los logins fallidos y bloqueados, y en Usuarios que cada persona tenga el rol que corresponde.
- Las claves de los servicios (Gmail, base de datos, cifrado, firma) no están en el código ni en GitHub, y se cambian si hay sospecha de que se filtraron.

4.8 Cuentas del servidor

No se trabaja como root (se usa `sudo`), una cuenta por persona, SSH solo con clave y se bloquean las cuentas que ya no se usan (scripts de S.O.).

5. Informe OWASP del sitio

SiGeRU se revisó contra el **OWASP Top 10 (2021)**: 8 de las 10 categorías están cubiertas, 1 parcial y 1 no aplica, y la revisión automática pasa 28 de 28 pruebas. Durante la revisión aparecieron 5 problemas, que ya están corregidos. El riesgo que queda es bajo y está en A05 (no hay política de contenido CSP) y A06 (las librerías se actualizan a mano).

Fecha: 06/11/2026 · **Alcance**: landing, frontend y las tres APIs, sobre `https://localhost` con los datos de prueba · **Método**: revisión del código + `seguridad/owasp/revision_owasp.sh`, que ataca al sistema con pedidos HTTP reales (accesos sin permiso, inyección SQL, fuerza bruta, archivos internos, cabeceras, cookies).

5.1 Resultado por categoría

Categoría OWASP	Cómo la cubre SiGeRU	Pruebas	Estado
A01 Control de acceso roto	`verificarRol()` en cada endpoint, el vecino solo ve sus incidencias, el admin no se cambia su rol	7 de 7 (401 sin sesión, 403 por rol)	Cubierto

Categoría OWASP	Cómo la cubre SiGeRU	Pruebas	Estado
A02 Fallas criptográficas	HTTPS TLS 1.2/1.3, bcrypt, AES-256-GCM, SHA-256, cookie `Secure`	4 de 4	Cubierto
A03 Inyección	Consultas preparadas en todos los modelos, ids solo numéricos, `limpiarTexto()` contra XSS	4 de 4 (`' OR '1'='1` no entra)	Cubierto
A04 Diseño inseguro	Bloqueo por intentos, verificación por correo + aprobación, límites de largo	1 de 1 (429 al 6to intento)	Cubierto
A05 Configuración de seguridad	Solo `api.php` accesible, sin listado de carpetas, cabeceras, errores sin detalles de PHP	8 de 8	Cubierto, falta CSP
A06 Componentes vulnerables	Leaflet 1.9.4 y PHPMailer 6.9.1 guardadas en el proyecto (sin CDN), sin vulnerabilidades conocidas que afecten el uso de SiGeRU	Revisión manual	Parcial
A07 Fallas de identificación y autenticación	Contraseñas fuertes, mismo error para correo inexistente, sesión nueva al entrar, vence a los 30 min	3 de 3	Cubierto
A08 Fallas de integridad	Respaldos firmados con RSA; no se deserializan datos del usuario (solo `json_decode`)	Prueba de firma con 1 byte cambiado	Cubierto
A09 Fallas de registro y monitoreo	Tabla `auditoria` + página Seguridad, logs de Apache y Fail2ban	1 de 1 (el ataque quedó registrado)	Cubierto
A10 Falsificación de pedidos del lado del servidor (SSRF)	El servidor no pide direcciones que mande el usuario (solo SMTP a Gmail, fijo)	Revisión de código	No aplica

5.2 Hallazgos y correcciones

#	Hallazgo	Categoría	Riesgo antes	Corrección	Estado
1	Los tests se podían correr desde el navegador y modificaban la base	A05	Alto	`.htaccess` con `Require all denied` en `tests/`	Corregido
2	`conexion.php`, modelos y `config_correo.php` se podían pedir directo	A05	Medio	Solo se permite `api.php`	Corregido
3	Ids no numéricos llegaban a los modelos (`?id=1 OR 1=1` devolvía el 1)	A03	Bajo (ya usaba consultas preparadas)	Las APIs responden 400 si el id no es un entero positivo	Corregido
4	El login aceptaba intentos sin límite	A07	Alto	Bloqueo a los 5 intentos	Corregido
5	El código de verificación y el teléfono estaban en texto en la base	A02	Medio	Hash SHA-256 y cifrado AES-256-GCM	Corregido

5.3 Riesgos que quedan y recomendaciones

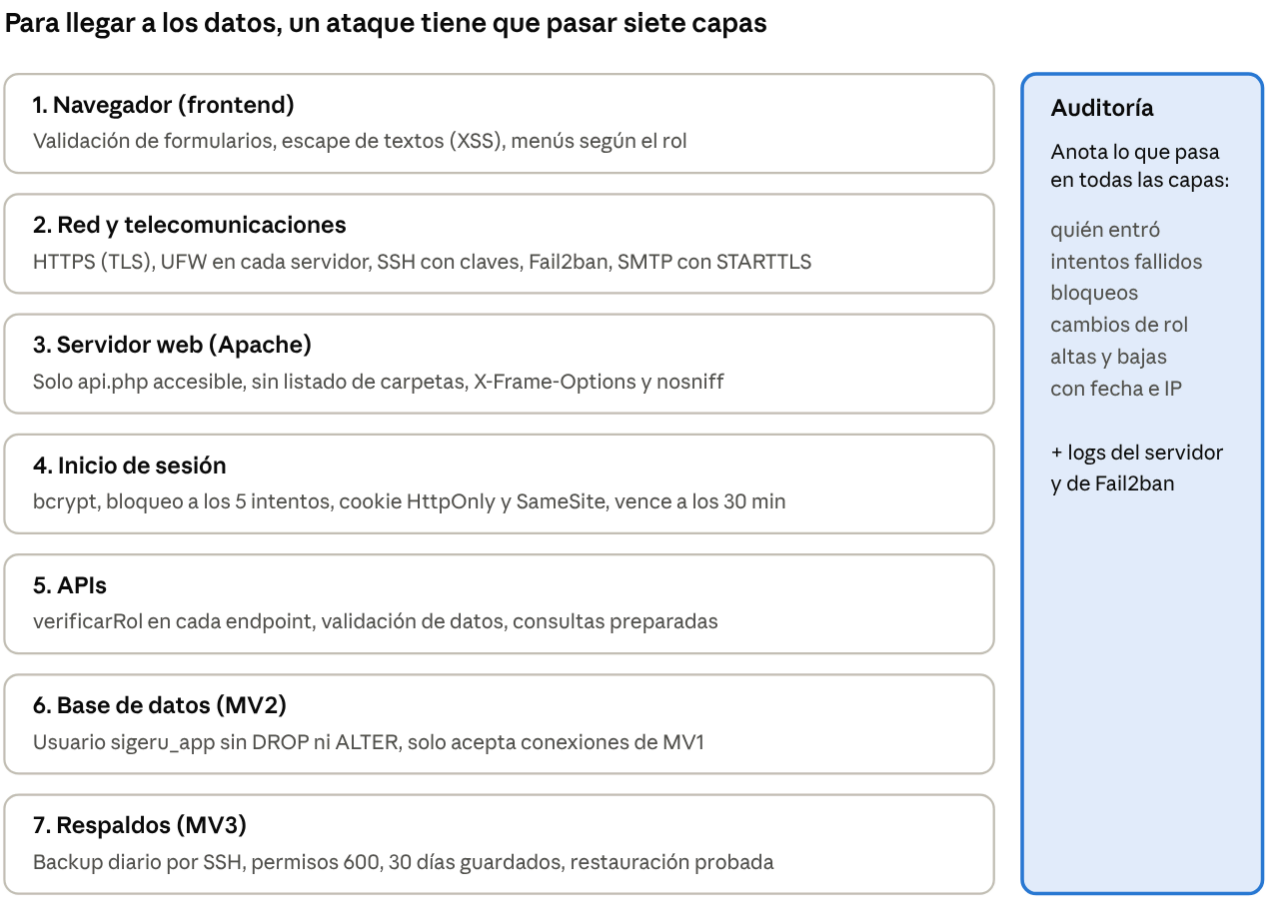
- **CSP (A05)**: el frontend usa `onclick` en el HTML, que una CSP estricta bloquearía. Se recomienda pasar esos eventos a `addEventListener` y agregar `Content-Security-Policy: default-src 'self'`.
- **Librerías (A06)**: revisar cada 3 meses las versiones de Leaflet y PHPMailer contra sus avisos de seguridad y actualizar PHPMailer a la última 6.x antes de instalar.
- **Doble factor (A07)**: agregar un código por correo para los administradores.

5.4 Cómo repetir el informe

Con los datos de prueba cargados: `bash seguridad/owasp/revision_owasp.sh https://localhost`. Salida del 06/11/2026: `Pasaron: 28 | Fallaron: 0`.

6. Estrategia: seguridad por capas

SiGeRU usa defensa en profundidad: cada capa tiene sus propios controles, así si una falla la siguiente frena el ataque. La auditoría atraviesa todas las capas y deja registrado lo que pasa.



defensa en profundidad · 7 capas + auditoría

Las capas 1, 3, 4 y 5 son de Programación Full Stack; la 2, 6 y 7 se configuran con los scripts de Administración de Sistemas Operativos.

7. Análisis de riesgos

Los riesgos más altos para SiGeRU son el robo de cuentas y la inyección de datos por las APIs, y los dos tienen controles implementados. Se calculó el riesgo como **probabilidad × impacto** (cada uno de 1 a 3): 6 o más es alto, 3 o 4 es medio, menos es bajo. El riesgo residual es el que queda después de aplicar los controles.

Amenaza	Prob.	Impacto	Riesgo	Control aplicado	Residual
Fuerza bruta contra el login de SiGeRU	3	3	Alto (9)	Bloqueo a los 5 intentos por 15 min, auditoría, contraseñas fuertes	Bajo
Inyección SQL por los formularios o las APIs	2	3	Alto (6)	Consultas preparadas (`prepare` + `bind_param`) en todos los modelos, validación en backend	Bajo

Amenaza	Prob.	Impacto	Riesgo	Control aplicado	Residual
Un rol hace algo que no le corresponde (escalada de privilegios)	2	3	Alto (6)	`verificarRol()` en cada endpoint, un admin no se cambia su propio rol, auditoría de cambios de rol	Bajo
XSS: alguien escribe código en una incidencia y se ejecuta en el navegador del operario	2	3	Alto (6)	`limpiarTexto()` escapa todo lo que se muestra, `nosniff`	Bajo
Robo o reuso de la sesión	2	3	Alto (6)	Cookie `HttpOnly` + `SameSite=Strict`, sesión nueva al entrar, vence a los 30 min	Bajo
Fuerza bruta contra SSH del servidor	3	3	Alto (9)	SSH con claves, Fail2ban, UFW (módulo de S.O.)	Bajo
Acceso directo a la base de datos	1	3	Medio (3)	MV2 separado, MySQL solo acepta a MV1, usuario `sigeru_app` sin DROP/ALTER	Bajo
Filtración de la contraseña de Gmail o de la base	2	2	Medio (4)	Credenciales fuera del código y del repositorio (`sigeru_correo.php` , `DB_PASS`)	Bajo
Pedidos falsos desde otra página (CSRF) o meter SiGeRU en un iframe (clickjacking)	1	2	Bajo (2)	`SameSite=Strict` , `X-Frame-Options: DENY`	Bajo
Registro de cuentas falsas	2	1	Bajo (2)	Verificación por Gmail + aprobación del administrador	Bajo
Pérdida de datos (falla, error humano, ransomware)	2	3	Alto (6)	Backups automáticos al servidor de respaldo MV3, retención	Medio
No saber quién hizo un cambio	3	2	Alto (6)	Tabla `auditoria` con usuario, IP y fecha	Bajo

El único riesgo que queda en medio es la pérdida de datos: depende de que los backups se prueben restaurando, algo que se hace a mano (ver sección 10).

8. Seguridad del software (implementada)

Todo lo de esta sección está en el repositorio y tiene tests automáticos. Cada control dice dónde está el código para que el docente lo pueda revisar.

8.1 Autenticación y registro

- Las contraseñas se guardan con `password_hash()` (bcrypt, con sal automática) y se comprueban con `password_verify()` . Nunca se guardan en texto ni se devuelven en las listas (`modelos/Usuario.php`).

- El vecino que se registra tiene que verificar su correo con un enlace de un solo uso (código de 32 caracteres hechos con `random_bytes`) y después un administrador aprueba la cuenta. Mientras tanto no puede entrar (403).
- El mensaje de error del login es siempre "Correo o contraseña incorrectos": no dice si el correo existe, así no se pueden adivinar cuentas.

8.2 Protección contra fuerza bruta (nuevo)

Cada intento fallido queda en la tabla `auditoria`. Si un correo junta **5 intentos fallidos en 15 minutos** (después de su último ingreso correcto), la API responde **429** y no deja seguir probando hasta que pasen los 15 minutos. Los dos números son parámetros (`intentos_login`, `minutos_bloqueo`) que el administrador cambia desde la página Parámetros, sin tocar código. El bloqueo es por correo: los demás usuarios pueden seguir entrando.

Es la misma idea que Fail2ban hace con SSH en el servidor, pero dentro de la aplicación.

8.3 Sesiones (nuevo)

- La cookie `SIGERU` es `HttpOnly` (JavaScript no la puede leer, así un XSS no la roba) y `SameSite=Strict` (no viaja en pedidos que vienen de otras páginas, eso frena el CSRF). Con HTTPS también se marca `Secure`.
- Al iniciar sesión se llama a `session_regenerate_id(true)`: el número de sesión cambia y uno viejo no sirve (evita la fijación de sesión).
- Si pasan **30 minutos sin usar el sistema**, la sesión se cierra sola y el frontend vuelve al login con el aviso. Las consultas automáticas de la campana de notificaciones no cuentan como uso, si no la sesión no vencería nunca con la pestaña abierta.

8.4 Autorización por roles

Cada endpoint de las tres APIs llama a `verificarRol([...])` antes de hacer nada (`seguridad.php`). Sin sesión responde 401 y con un rol sin permiso responde 403. El frontend además esconde los menús y botones que el rol no puede usar, pero eso es solo comodidad: la regla real está en el backend.

Rol	Puede
Administrador	Todo, más usuarios, roles, parámetros y la auditoría
Operario	Gestión de contenedores, camiones, recorridos, centros, repuestos, incidencias e informes

Rol	Puede
Recolector	Ver su recorrido y atender las incidencias de su cuadrilla
Vecino	Reportar incidencias y ver solo las suyas

Reglas extra: un administrador no se puede cambiar su propio rol ni borrarse a sí mismo (así nunca queda el sistema sin administrador), y un vecino que pide una incidencia ajena recibe 403.

8.5 Validación de datos (frontend y backend)

- **Frontend**: campos `required`, `minlength`, `maxlength`, `pattern` y tipos (`email`, `number`) en los formularios, y validación en JavaScript antes de mandar.
- **Backend**: se valida todo de nuevo porque el frontend se puede saltear (por ejemplo con Postman). Formato del correo con expresión regular, roles y estados contra una lista cerrada, números no negativos, descripción de incidencias de 10 a 500 caracteres.

8.6 Inyección SQL y XSS

- Todas las consultas con datos del usuario usan `mysqli::prepare` + `bind_param`: el dato nunca se mezcla con el SQL.
- Todo texto que viene de la base se muestra con `limpiarTexto()`, que cambia `<`, `>`, `&` y las comillas por entidades HTML. Si un vecino escribe `<script>` en una incidencia, el operario lo ve como texto.

8.7 Auditoría (nuevo)

La tabla `auditoria` guarda fecha, usuario, correo, acción, detalle e IP de: inicio y cierre de sesión, login fallido, login bloqueado, registro, aprobación o rechazo, usuario creado, modificado o borrado, cambio de rol, cambio de contraseña y cambio de parámetros. El administrador la ve en la página **Seguridad**, con contadores, filtro por acción y exportación a Excel. Solo el administrador puede leerla (403 para los demás).

8.8 Configuración del servidor web (nuevo)

- Desde el navegador solo se puede abrir `api.php` de cada API. `conexion.php`, los modelos, `config_correo.php` y la carpeta `tests` devuelven **403** (`.htaccess`). Antes alguien podía correr los tests desde el navegador y cambiar la base.
- `Options -Indexes`: no se listan los archivos de las carpetas.

- Cabeceras en todas las respuestas: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (nadie puede meter SiGeRU en un iframe), `Referrer-Policy: same-origin` y `Cache-Control: no-store` en las APIs (los datos personales no quedan en el caché).

9. Seguridad en telecomunicaciones

Cada servidor abre solo los puertos que necesita y solo para quien los necesita: desde Internet se llega únicamente a la web por HTTPS. La red sigue el esquema de la 2da entrega (MV1 API 192.168.1.200, MV2 BD 192.168.1.201, MV3 Respaldo 192.168.1.202).

9.1 Reglas del firewall (UFW) por servidor

Política por defecto en los tres: `deny incoming`, `allow outgoing`.

Servidor	Puerto	Desde	Para qué
MV1 – API	443/tcp	Cualquiera	SiGeRU por HTTPS
MV1 – API	80/tcp	Cualquiera	Solo redirige a HTTPS
MV1 – API	22/tcp	Red de administración	SSH con clave
MV2 – BD	3306/tcp	Solo 192.168.1.200	MySQL, únicamente las APIs
MV2 – BD	22/tcp	Red de administración	SSH con clave
MV3 – Respaldo	22/tcp	192.168.1.201 y red de administración	Recibir backups por rsync/scp sobre SSH

Ejemplo en MV2:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.1.200 to any port 3306 proto tcp
sudo ufw allow from 192.168.1.0/24 to any port 22 proto tcp
sudo ufw enable
```

En MySQL además se pone `bind-address = 192.168.1.201` (no escucha en otras interfaces) y el usuario de las APIs se crea como `'sigeru_app'@'192.168.1.200'`, así aunque alguien pase el firewall no puede entrar desde otra IP.

9.2 HTTPS (TLS)

Sin HTTPS la contraseña del login y la cookie de sesión viajarían en texto plano y cualquiera en la misma Wi-Fi las podría leer. Está implementado con un certificado de la CA EcoSync, TLS 1.2 y 1.3 y redirección del puerto 80: el detalle y la evidencia están en la sección 3 y la configuración en `seguridad/ssl/sigeru-ssl.conf`.

Cuando hay HTTPS la API marca sola la cookie como `Secure` (no se manda nunca por HTTP). En XAMPP, para desarrollo, se trabaja con `http://localhost`.

9.3 SSH

Va en el módulo de seguridad de los scripts de S.O.: `PermitRootLogin no`, `PasswordAuthentication no` (solo claves), `MaxAuthTries 3`, `AllowUsers` con las cuentas de administración y Fail2ban sobre `sshd`.

9.4 Correo (Gmail)

Los correos de verificación y aprobación salen por `smtp.gmail.com:587` con **STARTTLS** (PHPMailer), o sea cifrados. Se usa una contraseña de aplicación de Google, no la contraseña de la cuenta, y se puede revocar sola sin tocar la cuenta.

9.5 Contenedores (Docker)

En `docker-compose.yml` el contenedor `web` llega a la base por la red interna de Docker. Para desarrollo la base también se publica en el puerto 3307 (para conectarse con un cliente de MySQL); en el servidor de producción se saca esa línea `ports` para que la base no quede expuesta, igual que phpMyAdmin (8081), que solo se usa en desarrollo.

10. Base de datos y respaldos

Las APIs entran a MySQL con un usuario que solo puede leer y escribir filas, y los respaldos se copian a otro servidor. Así un error o un ataque a MV1 no puede borrar la estructura de la base, y si MV2 se pierde quedan los backups en MV3.

10.1 Mínimo privilegio en MySQL

El script `sigeru-base-datos/seguridad/usuario_aplicacion.sql` crea el usuario `sigeru_app`, que solo puede conectarse desde 192.168.1.200 y solo tiene `SELECT, INSERT, UPDATE, DELETE` sobre la base `sigeru`. No tiene `DROP`, `ALTER`, `CREATE` ni `GRANT`. El usuario `root` queda solo para el administrador del servidor y no se usa desde las APIs.

La contraseña no está en el código: `conexion.php` la lee de las variables de entorno `DB_USER` y `DB_PASS` del servidor. En XAMPP, si no están, usa root sin contraseña como siempre.

10.2 Integridad de los datos

- Claves foráneas con `ON DELETE` pensado en cada caso (por ejemplo, si se borra un usuario su auditoría queda con `id_usuario` en NULL, no se pierde).
- `ENUM` para estados, prioridades y roles: la base no acepta un valor que no exista.
- Las actualizaciones de estructura van en `sigeru-base-datos/actualizaciones/` y nunca borran datos (la 002 crea la auditoría).

10.3 Respaldos

Qué	Cómo	Cada cuánto	Dónde
Base `sigeru` completa	`mysqldump --single-transaction` comprimido con gzip	Todas las noches (cron)	MV2 y copia a MV3 por SSH
Código de las aplicaciones	Repositorio en GitHub	Cada commit	GitHub
Configuración del servidor	Script de respaldo de S.O.	Semanal	MV3

En MV3 los archivos tienen permisos `600` y dueño el usuario de backups, y se guardan los últimos 30 días. Una vez por mes se prueba restaurar un backup en una base de prueba: un backup que nunca se restauró no se sabe si sirve.

11. Plan de respuesta a incidentes

Si pasa algo, el equipo sigue cinco pasos en orden; el coordinador decide y anota todo. La página Seguridad y los logs del servidor son la primera fuente de información.

1. **Detectar**: la página Seguridad muestra muchos "Login fallido" o "Login bloqueado", Fail2ban banea IPs, o un usuario avisa algo raro (un rol cambiado, datos que no cargó).
2. **Contener**: bloquear la cuenta afectada (estado Rechazado o cambio de contraseña), banear la IP con `ufw deny from <IP>` y, si es grave, cortar el acceso público a MV1.
3. **Investigar**: filtrar la auditoría por correo o acción y exportarla a Excel; revisar `/var/log/auth.log`, `/var/log/apache2/` y los registros de Fail2ban para saber qué se hizo, cuándo y desde dónde.

4. **Recuperar**: corregir lo que se cambió; si se perdieron o dañaron datos, restaurar el último backup bueno de MV3; cambiar las credenciales que pudieron quedar expuestas.
5. **Aprender**: escribir qué pasó y qué control faltó, y agregarlo (por ejemplo, bajar `intentos_login` o cerrar un puerto).

Ejemplo: si la auditoría muestra 40 intentos fallidos en una hora contra `admin@...` desde la misma IP, el bloqueo ya frenó el ataque; igual se banea esa IP en UFW y se le pide al administrador que cambie su contraseña.

12. Pruebas de seguridad y evidencias

Los 15 casos de esta tabla pasaron. Los de la API están automatizados en `sigeru-api-usuarios/tests/test_api.php` (sección "Seguridad"), que pasa 60 de 60; los de la interfaz se probaron en el navegador.

#	Caso de prueba	Datos	Esperado	Resultado
1	Contraseña corta en el registro	`123`	400	OK
2	Contraseña sin números	`soloLetras`	400 "letras y números"	OK
3	Contraseña sin letras	`12345678`	400	OK
4	5 contraseñas equivocadas seguidas	correo de prueba, `mala1` … `mala5`	401 cada una	OK
5	Sexto intento	`otra-mala`	429 y bloqueo de 15 min	OK
6	Otro correo durante el bloqueo	`vecino@sigeru.uy`	401 normal (no bloqueado)	OK
7	Operario pide la auditoría	sesión de operario	403	OK
8	Admin pide los logins fallidos	filtro "Login fallido"	Los 5 intentos con correo e IP	OK

#	Caso de prueba	Datos	Esperado	Resultado
9	Cambio de rol queda registrado	vecino → Operario → Administrador	Detalle "pasó de Operario a Administrador" a nombre del admin	OK
10	Abrir `conexion.php` desde el navegador	GET directo	403	OK
11	Correr los tests desde el navegador	GET `tests/test_api.php`	403	OK
12	Sesión sin usar más de 30 min	se simuló un último uso viejo	401 "Tu sesión se cerró..."	OK
13	Cookie de sesión	cabecera `Set-Cookie`	`HttpOnly; SameSite=Strict`	OK
14	Cabeceras de seguridad	cabeceras del frontend y la API	`X-Frame-Options: DENY`, `nosniff`	OK
15	Vecino entra a la página Seguridad	sesión de vecino	No ve el menú y la API responde 403	OK

Además siguen pasando todos los tests de las entregas anteriores que cubren seguridad: 401 sin sesión, 403 por rol en las tres APIs, el vecino solo ve sus incidencias, el admin no se cambia su propio rol y las contraseñas no se guardan en texto.

Cómo repetir las pruebas: con XAMPP prendido, `php sigeru-api-usuarios/tests/test_api.php` desde la carpeta del proyecto.

Evidencias para agregar en la entrega: captura de la página Seguridad con los intentos fallidos, captura del mensaje de bloqueo en el login, la salida de los tests, y de S.O. `ufw status verbose`, `fail2ban-client status sshd` y la configuración de SSH.

13. Registro de implementaciones y conclusión

De los 19 controles, 17 están implementados y probados; UFW y SSH con Fail2ban se aplican al instalar los servidores con los scripts de S.O.

Control	Capa	Estado	Evidencia
Hash bcrypt de contraseñas	Aplicación	Implementado	Tests de modelos
Política de contraseñas	Aplicación	Implementado (3ra)	Casos 1 a 3
Bloqueo por intentos fallidos	Aplicación	Implementado (3ra)	Casos 4 a 6
Auditoría + página Seguridad	Aplicación	Implementado (3ra)	Casos 7 a 9
Sesión segura y con vencimiento	Aplicación	Implementado (3ra)	Casos 12 y 13
Autorización por roles	Aplicación	Implementado	Tests de las 3 APIs
Consultas preparadas y escape XSS	Aplicación	Implementado	Código de modelos y vistas
Verificación por correo + aprobación	Aplicación	Implementado	Tests de registro
Solo `api.php` accesible + cabeceras	Servidor web	Implementado (3ra)	Casos 10, 11 y 14
Usuario MySQL con mínimo privilegio	Base de datos	Script listo (3ra)	`usuario_aplicacion.sql`
Credenciales fuera del código	Aplicación	Implementado	Repositorio sin contraseñas
SMTP con STARTTLS	Telecomunicaciones	Implementado	`correo.php`
UFW por servidor	Telecomunicaciones	Al instalar (scripts S.O.)	`ufw status`
SSH con claves + Fail2ban	Telecomunicaciones	Al instalar (scripts S.O.)	Configuración y estado

Control	Capa	Estado	Evidencia
HTTPS con certificado	Telecomunicaciones	Implementado (3ra)	Sección 3.3
Cifrado AES-256-GCM del teléfono	Base de datos	Implementado (3ra)	Tests de modelos (sección 2)
Hash SHA-256 del código de verificación	Aplicación	Implementado (3ra)	Tests de modelos y de la API
Revisión OWASP automática	Todas	Implementado (3ra)	28 de 28 (sección 5)
Backups con hash, cifrado y firma a MV3	Infraestructura	Scripts probados (3ra), cron al instalar	Prueba de firma con 1 byte cambiado

Mejoras a futuro: doble factor (código por correo) para administradores, recuperar la contraseña por correo, una política de contenido (CSP) en el frontend, y avisar al administrador cuando una cuenta se bloquea.

Conclusión: la seguridad de SiGeRU no depende de una sola medida. Si alguien adivina una contraseña, el bloqueo lo frena; si entra igual, su rol limita lo que puede hacer y la auditoría deja registrado qué hizo; si ataca la base, el firewall y el usuario con mínimo privilegio lo frenan; y si algo se pierde, están los respaldos. Eso es lo que pide la letra: estrategias en cada capa que se complementan.